

○ 2024.12.02 ○

“주행2”

SLAM 기반 로봇 주행 환경 장애물 인식 모델
개발

김현아

|

장석환

|

홍유진

• INDEX

목차

01

과제 1

02

과제 2

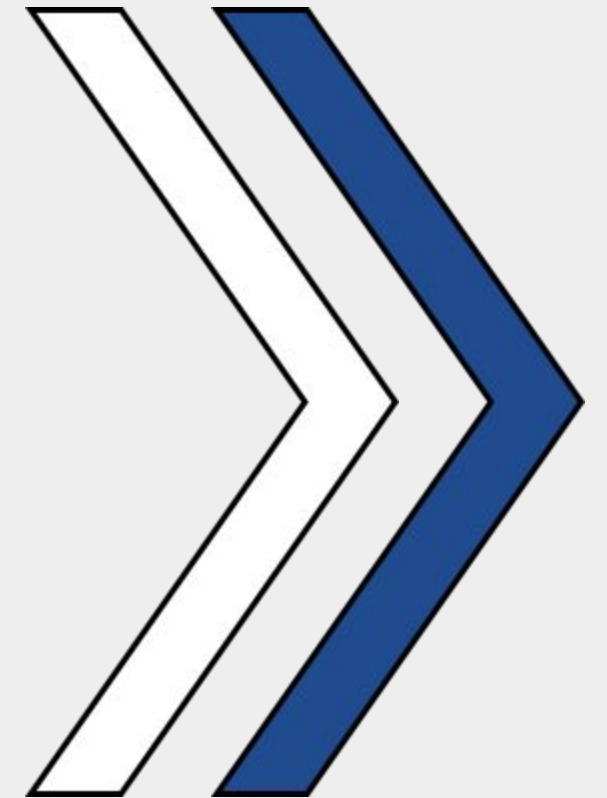
03

과제 3

04

시연

과제 1



- 맵 로직
- 튜닝 전략

01

맵 로직



01

맵 로직 - 맵 콜백 & 전체 로직

```
def map_callback(self, msg):
    # Extract relevant map information
    width = msg.info.width
    height = msg.info.height
    resolution = msg.info.resolution
    origin_x = msg.info.origin.position.x
    origin_y = msg.info.origin.position.y

    self.unknown_cells.clear()
    step = 4 # 탐색 속도를 높이기 0.001 위해 스킵 간격 설정

    for y in range(1, height - 1, step):
        for x in range(1, width - 1, step):
            index = x + (y * width)
            if msg.data[index] == -1 and self.is_near_free_space(x, y, width, msg.data):
                world_x = origin_x + (x * resolution)
                world_y = origin_y + (y * resolution)
                self.unknown_cells.append((world_x, world_y))

    if self.unknown_cells and self.is_time_to_set_new_goal():
        self.send_goal(msg) # 맵 데이터를 전달
    elif not self.unknown_cells:
        self.get_logger().info('미탐색 영역이 없습니다.')
        self.publish_exploration_done() # 탐색 완료 퍼블리시
```

맵 데이터를 분석하여 미탐사 영역 탐지

1. 미탐사 영역 확인
2. 미탐사 셀이 자유공간 근처에 있는지 확인
3. 미탐사 셀의 좌표를 저장
- 4-1. 미탐사 영역이 있으면 목표 지점 설정
- 4-2. 미탐사 영역이 없다면 탐색 완료

01

맵 로직 – 완료 조건 추가 & 로봇 위치 판단

```
def update_robot_position(self, robot_x, robot_y):  
    """  
    로봇 위치 갱신 및 변화 확인.  
    """  
  
    self.robot_position = (robot_x, robot_y)  
    if self.last_position is None:  
        self.last_position = self.robot_position  
        self.last_position_update_time = self.get_clock().now()  
        return  
  
    # 위치 변화 계산  
    distance = math.sqrt((robot_x - self.last_position[0])**2 + (robot_y - self.last_position[1])**2)  
  
    # 0.05m 이상 움직이면 위치 갱신 0.05m 이상 이동했는지 확인  
    if distance > 0.05:  
        self.last_position = self.robot_position  
        self.last_position_update_time = self.get_clock().now()  
    else:  
        current_time = self.get_clock().now()  
        elapsed_time = (current_time - self.last_position_update_time).nanoseconds / 1e9  
        loading_time = (current_time - self.robot_time).nanoseconds / 1e9  
        if elapsed_time > 15 and loading_time > 120.0:  
            self.publish_exploration_done() # 탐색 완료 퍼블리시  
            time.sleep(5)
```

탐색 완료 조건

01

맵 로직 - 자유 공간 판단

```
def is_near_free_space(self, x, y, width, data):  
    neighbors = [  
        (x - 1, y), (x + 1, y),  
        (x, y - 1), (x, y + 1),  
        (x - 1, y - 1), (x - 1, y + 1),  
        (x + 1, y - 1), (x + 1, y + 1)  
    ]  
  
    for nx, ny in neighbors:  
        index = nx + (ny * width)  
        if 0 <= index < len(data) and data[index] == 0: # 자유 공간 조건  
            return True  
    return False
```

주변 8개 픽셀 확인

01

맵 로직 - 벽 판단

```
def is_wall(self, x, y, map_info, map_data):  
    """  
    지정된 좌표가 벽인지 확인.  
    """  
    # 월드 좌표를 맵 좌표로 변환  
    map_x = int((x - map_info.origin.position.x) / map_info.resolution)  
    map_y = int((y - map_info.origin.position.y) / map_info.resolution)  
  
    # 좌표가 맵 범위 내에 있는지 확인  
    if map_x < 0 or map_y < 0 or map_x >= map_info.width or map_y >= map_info.height:  
        return True # 맵 범위 밖은 벽으로 간주  
  
    # 1D index 계산  
    index = map_x + (map_y * map_info.width)  
  
    # 벽 여부 확인 (OccupancyGrid 데이터에서 100은 벽을 나타냄)  
    if 0 <= index < len(map_data) and map_data[index] == 100:  
        return True  
  
    return False
```


01

맵 로직 - 이동 위치 선택

```
def find_frontier_clusters(self):
    """
    DBSCAN 클러스터링을 사용해 프론티어를 그룹화.
    """
    unknown_array = np.array(self.unknown_cells)
    clustering = DBSCAN(eps=0.5, min_samples=5).fit(unknown_array)

    clusters = {}
    for i, label in enumerate(clustering.labels_):
        if label not in clusters:
            clusters[label] = []
        clusters[label].append(self.unknown_cells[i])
    return clusters
```

```
def select_best_frontier(self, clusters, robot_x, robot_y):
    """
    가장 큰 크기이거나 로봇과 가장 가까운 프론티어 선택.
    """
    best_frontier = None
    max_size = 0
    min_distance = float('inf')

    for label, points in clusters.items():
        if label == -1: # 노이즈 클러스터 무시
            continue

        size = len(points)
        center_x = sum(p[0] for p in points) / size
        center_y = sum(p[1] for p in points) / size
        distance = math.sqrt((center_x - robot_x)**2 + (center_y - robot_y)**2)

        if size > max_size or (size == max_size and distance < min_distance):
            max_size = size
            min_distance = distance
            best_frontier = (center_x, center_y)

    return best_frontier
```

01

맵 로직 - 목표 전송

```
def send_goal(self, msg):
    if self.robot_position is None:
        self.get_logger().info("로봇 위치 정보가 없습니다. 목표 전송을 중단합니다.")
        return

    robot_x, robot_y = self.robot_position
    clusters = self.find_frontier_clusters()
    best_frontier = self.select_best_frontier(clusters, robot_x, robot_y)

    if best_frontier is None:
        self.get_logger().info('적절한 프론티어를 찾을 수 없습니다.')
        return

    target_x, target_y = best_frontier

    goal = PoseStamped()
    goal.header.frame_id = 'map'
    goal.header.stamp = self.get_clock().now().to_msg()

    if not self.is_wall(target_x, target_y, msg.info, msg.data) and self.is_within_limit(robot_x, robot_y, target_x, target_y):
        goal.pose.position.x = target_x
        goal.pose.position.y = target_y
        goal.pose.orientation.w = 1.0
        self.goal_pub.publish(goal)
        self.get_logger().info(f'목표 지점 전송: x={target_x}, y={target_y}')
    else:
        self.get_logger().info('목표 지점이 벽이거나 너무 멀어서 전송하지 않음.')
```


02 튜닝 전략 (nav2.yaml)

```
PathAlign.scale: 48.0 #32.0 >> 48.0
PathAlign.forward_point_distance: 0.2 # 0.2 >> 0.1
GoalAlign.scale: 24.0
GoalAlign.forward_point_distance: 0.2 # 0.2 >> 0.1
PathDist.scale: 48.0 #32.0 >> 48.0
```

PathAlign.scale: 경로 정렬 민감도 감소

PathAlign.forward_point_distance: 정렬기준점 앞쪽으로 설정

PathDist.scale: 경로 이탈 보정 민감도 감소

GoalAlign.scale: 목표 정렬 민감도를 조정

GoalAlign.forward_point_distance: 정렬 기준 거리 조정

“
controller_server - FollowPath
”

02 튜닝 전략 (nav2.yaml)

```
width: 1 #3 >> 1
height: 1 #3 >> 1
resolution: 0.02 #0.06 >> 0.02
robot_radius: 0.135 #0.175 >> 0.135
cost_scaling_factor: 2.0 #4.0 >> 2.0
inflation_radius: 0.35 #0.45 >> 0.35
```

width: 탐지 범위 축소

height: 탐지 범위 축소

resolution: 비용 지도 정밀도 증가

robot_radius: 로봇 반경 축소

cost_scaling_factor: 장애물 비용 완화

inflation_radius: 장애물 안전 거리 축소

“

Local Stamp & Global Stamp

”

02 튜닝 전략 (nav2.yaml)

```
raytrace_max_range: 2.5 #3.0 >> 2.5  
raytrace_min_range: 0.0  
obstacle_max_range: 2.0 #2.5 >> 2.0
```

“
Local Stamp & Global Stamp
”

Raytrace_max_range : 레이저 스캔 탐지 최대거리 감소

obstacle_max_range : 레이저 스캔 데이터 이용 최대 거리 감소

02 튜닝 전략 (nav2.yaml)

```
tolerance: 0.01 #1.0e-10 >> 0.01
```

“

smoother_server

”

Tolerance : 경로 조정 허용 오차 상향

02 튜닝 전략 (slam.yaml)

```
resolution: 0.01 #0.05 >> 0.01  
scan_buffer_size: 30 #20 >> 30
```

“

”

ROS & Global params

Resolution : 해상도 측정 값 증가

Scan_buffer_size : 스캔 데이터 저장 크기 증가

02 튜닝 전략 (slam.yaml)

```
loop_search_space_dimension: 5.0 #8.0 >> 5.0  
loop_search_space_resolution: 0.05 # 0.01 >> 0.05  
loop_search_space_smear_deviation: 0.03 # 0.1 >> 0.3
```

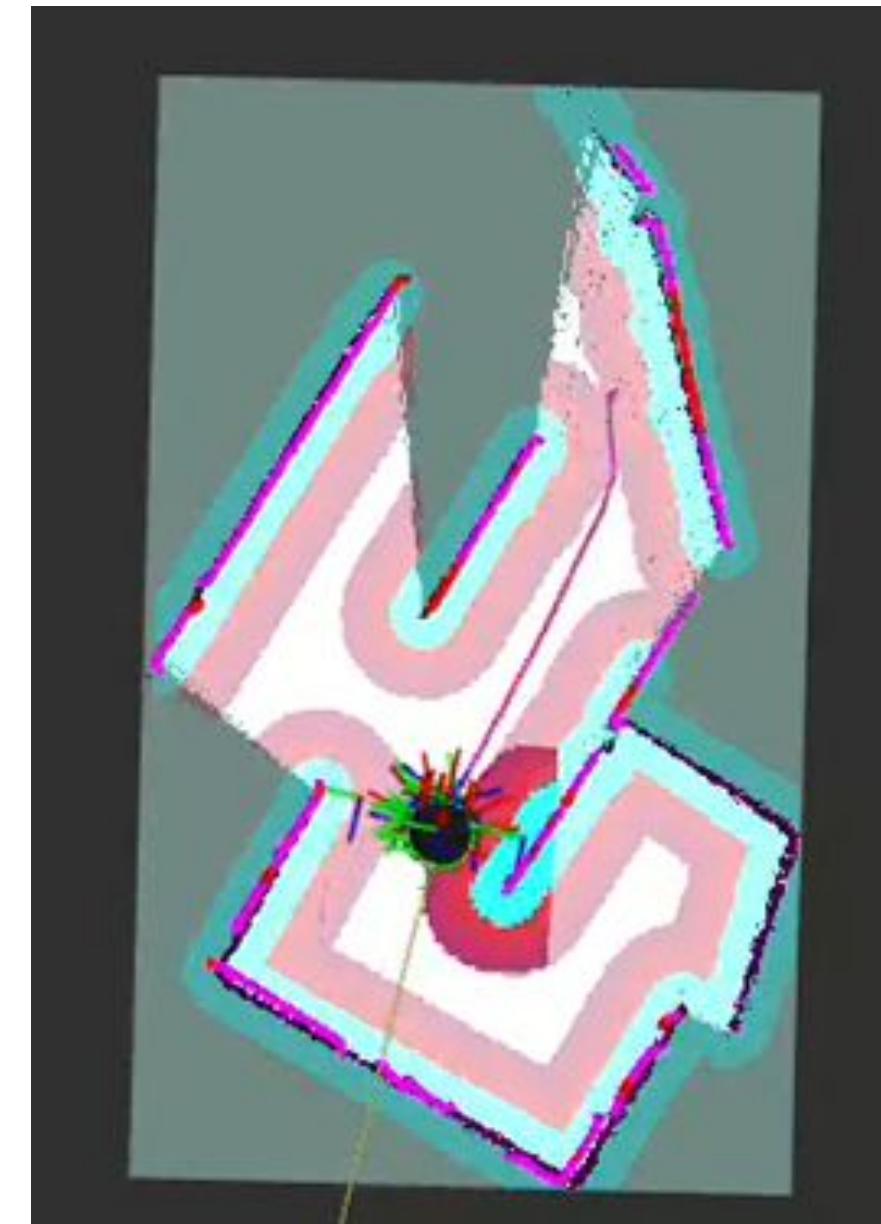
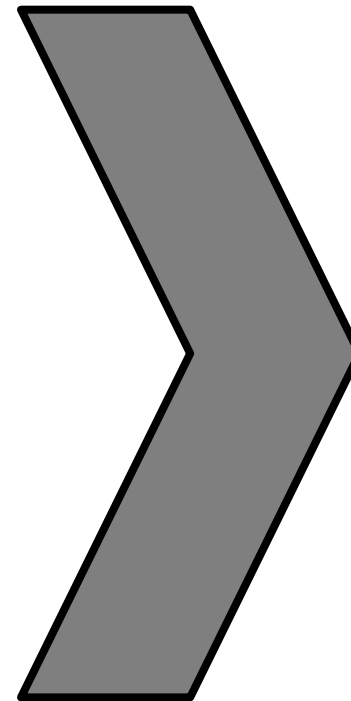
“

”

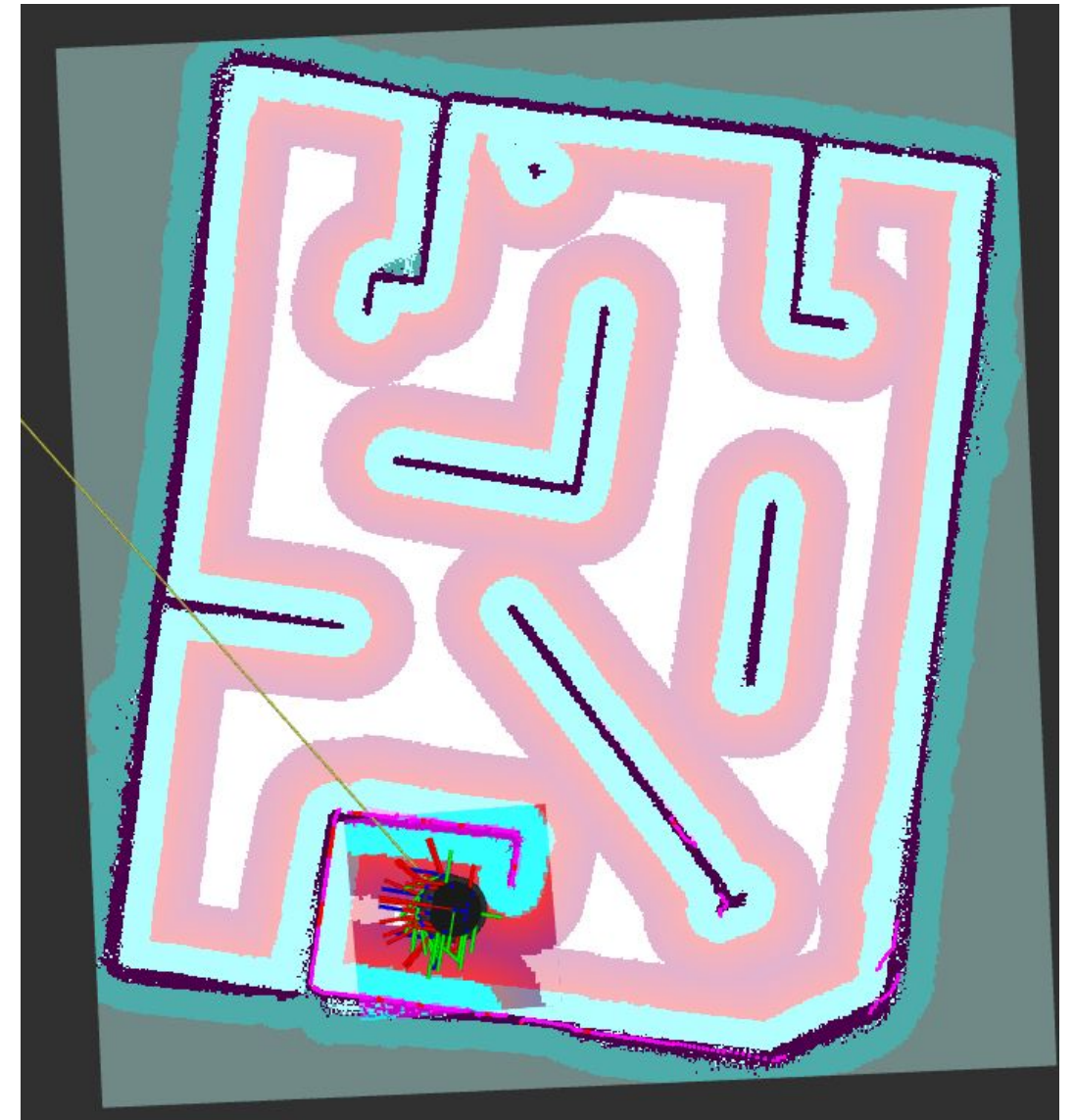
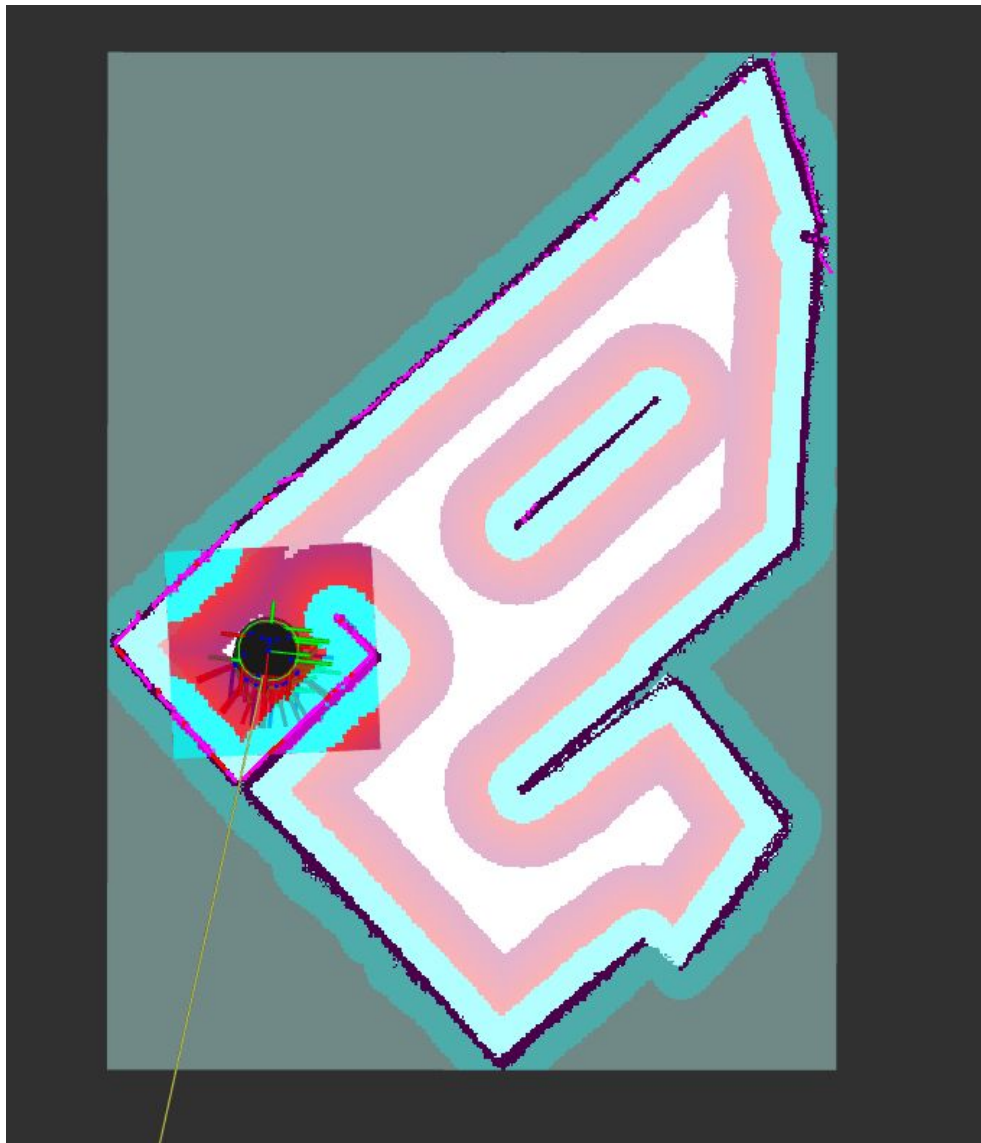
Loop Closure Parameters

loop_search_space_dimension : 루프 클로저 판단 공간 크기
loop_search_space_resolution : 격자 해상도 하락
loop_search_space_smear_deviation : 노이즈 허용 범위 감소

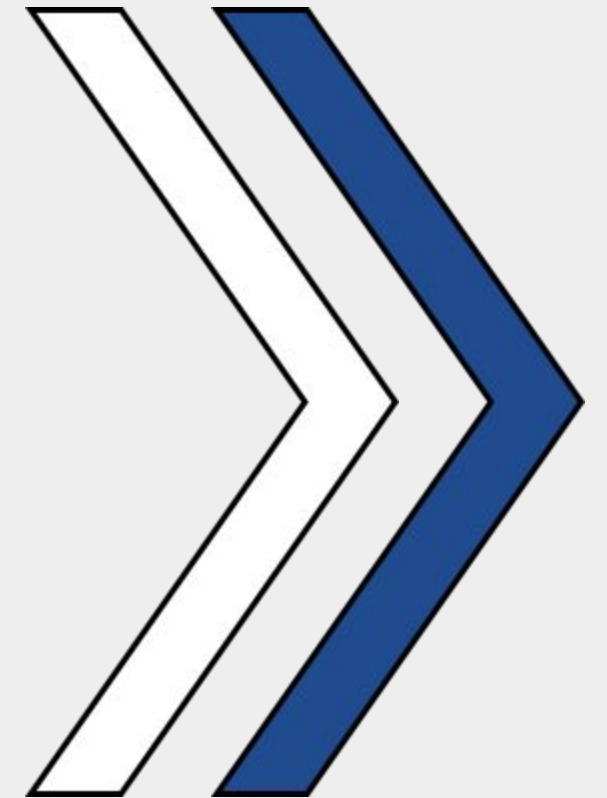
03 매핑 변화



04 모의 테스트 결과



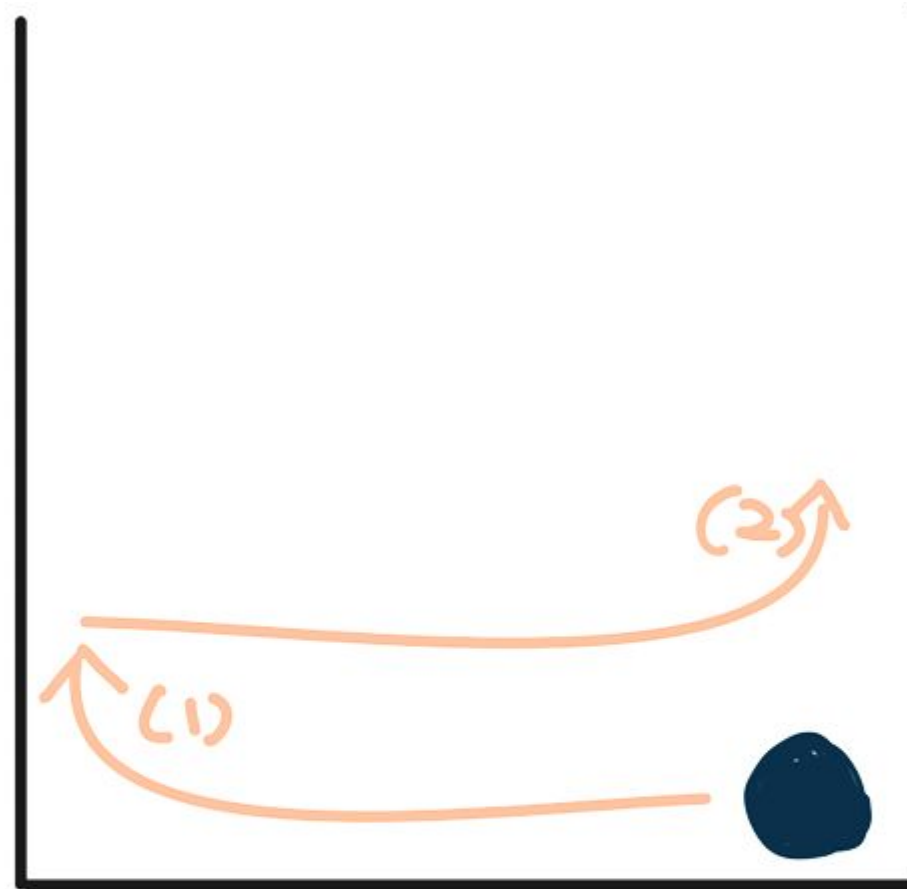
과제 2



- 가정 로직
- 코드 설명

01

코드 진행 방식 - 가정



(1)
wall_follow_left

(2)
wall_follow_right

02 가정 로직

```
def exploration_done_callback(self, msg):
    self.get_logger().info(f"Received /exploration_done message: {msg.data}")
    if msg.data and not self.wall_following_active:
        self.wall_following_active = True
        self.start_wall_following_timer()

def start_wall_following_timer(self):
    self.get_logger().info("탐색 완료 신호를 받았습니다. 벽 따라가기를 시작합니다.")
    if self.timer is None:
        self.timer = self.create_timer(5.0, self.send_wall_follow_goal, callback_group=self.callback_group)
```

1. 과제1 맵 로직으로 매핑이 끝났다면 /exploration_done이라는 매핑 완료 신호를 Bool 형식으로 전송
2. True 신호 수신 시 청소 로직 시작

02

가정 로직 – wall_follow 요청

```
def send_wall_follow_goal(self):  
    if not self.action_client.wait_for_server(timeout_sec=5.0):  
        self.get_logger().error('WallFollow 액션 서버를 찾을 수 없습니다.')  
        return  
  
    self.get_logger().info("Sending wall follow goal...")  
    goal_msg = WallFollow.Goal()  
    goal_msg.follow_side = self.follow_side  
    goal_msg.max_runtime = Duration(sec=10, nanosec=0) # 10초 동안 실행  
  
    self.get_logger().info(f'WallFollow 시작: 방향={"왼쪽" if self.follow_side == 1 else "오른쪽"}')  
    self.action_client.send_goal_async(goal_msg, self.goal_response_callback)
```

5초 이내 통신이 완료되었다면 벽의 방향을 받고 청소 시작

10초 동안 한 방향 유지

02 가정 로직 – wall_follow 요청

```
def result_callback(self, future):  
    try:  
        result = future.result().result  
        self.get_logger().info('WallFollow 액션 완료.')  
        self.retry_count = 0  
        self.follow_side = -self.follow_side  
        self.send_wall_follow_goal() # 다음 벽 따라가기 목표 전송  
    except Exception as e:  
        self.get_logger().error(f"Result callback에서 오류 발생: {str(e)}")  
        self.handle_failure()
```

액션이 끝났다면, 방향을 바꿔 다음 벽 따라가기로 변경

02

가정 로직 - 추가 사항

```
def map_callback(self, msg):
    """맵 데이터를 수신하고 numpy 배열로 저장"""
    self.map_data = np.array(msg.data).reshape((msg.info.height, msg.info.width))
    self.visited = np.zeros_like(self.map_data)
    self.grid_resolution = msg.info.resolution # 맵 해상도 설정
    self.get_logger().info(f"맵 데이터 수신 완료: {self.map_data.shape}, 해상도: {self.grid_resolution}")

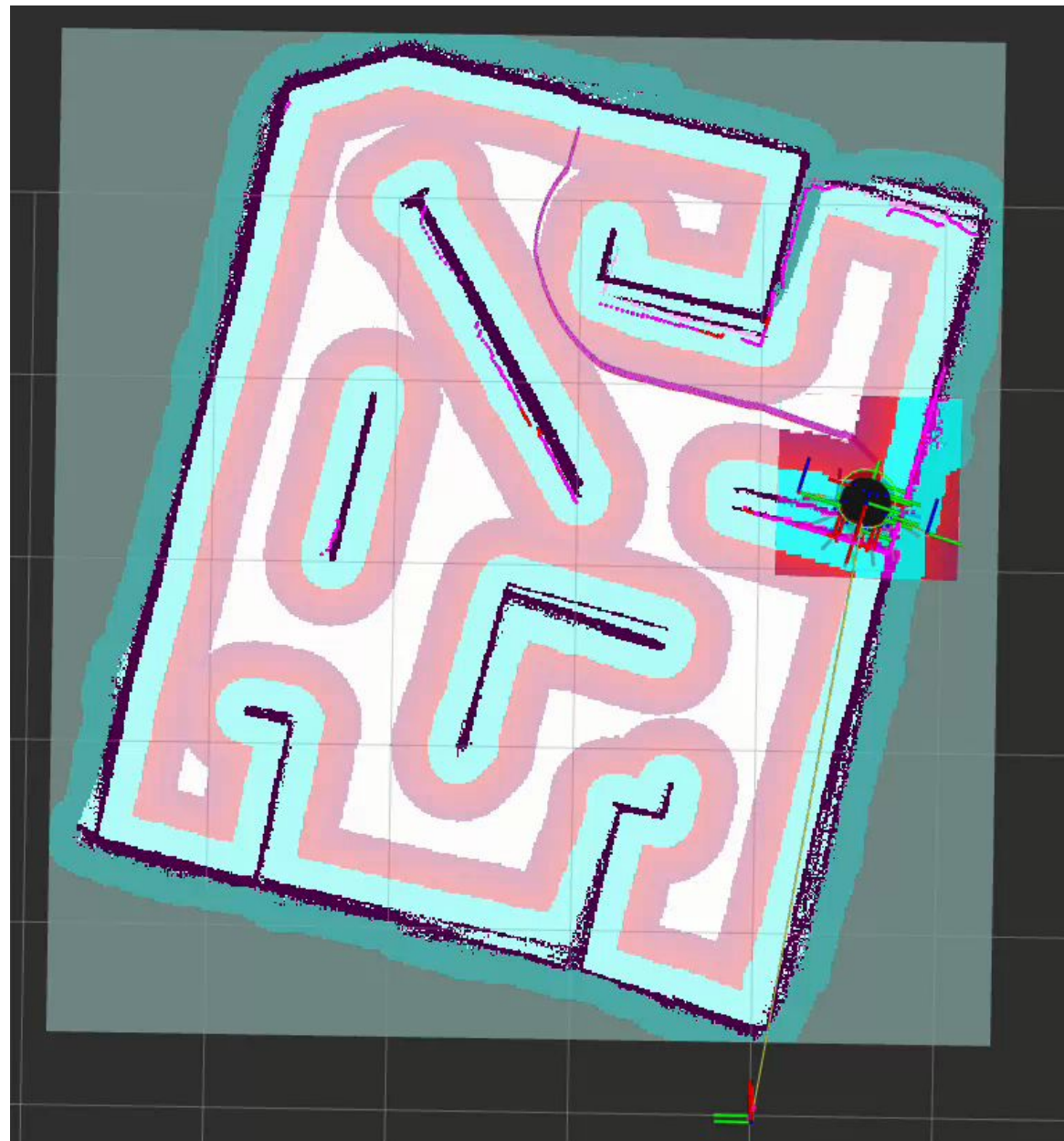
def odom_callback(self, msg):
    """로봇의 현재 위치를 업데이트"""
    self.current_pose = msg.pose.pose
    self.get_logger().info(f"현재 위치: x={self.current_pose.position.x}, y={self.current_pose.position.y}")

def plan_coverage(self):
    """맵 전체를 커버하기 위한 경로 계획"""
    if self.map_data is None or self.visited is None:
        self.get_logger().info("맵 데이터를 기다리는 중...")
    return
```

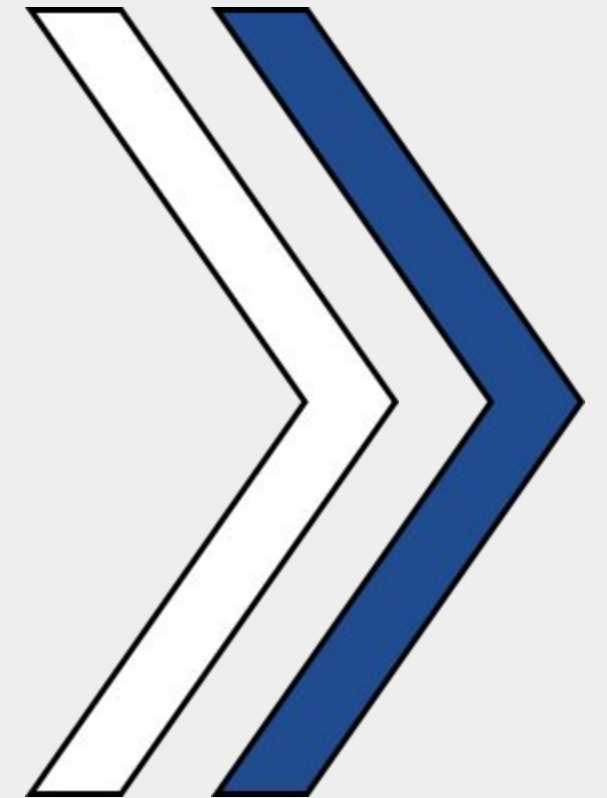
맵 데이터 수신 후 경로 계획

03

모의 테스트 결과



과제 3



- 코드 설명

01

코드 설명 – ORB 생성

```
def orb_tracking(video_file):  
    # ORB 객체 생성: 최대 특징점 수와 매개변수 조정  
    orb = cv2.ORB_create(nfeatures=2000, scaleFactor=1.1, nlevels=12)  
  
    # 최대 특징점 개수(nfeatures): 탐지할 최대 특징점 개수 설정  
    # 척도 피라미드 레벨(nlevels): 다중 해상도에서 특징점을 탐지하기 위한 레벨 수  
    # 특징점 품질 임계값(scaleFactor): 이미지 피라미드 축소 비율, 낮을수록 더 많은 특징점을 탐지  
  
    vid = VideoFrameSimulator(video_file)  
  
    # 추적을 위한 변수 초기화  
    prev_keypoints = None  
    prev_descriptors = None  
    prev_points = None  
  
    # 특징 매칭을 위한 BFMatcher 객체 생성  
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
```

ORB 객체 규칙 설정 후
ORB 및 BF 객체 생성

01

코드 설명 – ORB 변환 & 추적

```
for frame in vid.iterate_frame():
    # ORB를 사용하기 위해 프레임을 그레이스케일로 변환
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # ORB 특징점 탐지 및 계산
    keypoints, descriptors = orb.detectAndCompute(gray, None)

    # 현재 프레임의 복사본 생성 (추적 결과 표시용)
    tracked_frame = frame.copy()

    # 특징점을 추적하여 초록색 점으로 표시
    if prev_keypoints is not None and prev_descriptors is not None:
        # BFMatcher를 사용하여 특징 매칭
        matches = bf.match(prev_descriptors, descriptors) # 유사점 매칭 (해밍 거리)
        matches = sorted(matches, key=lambda x: x.distance) # 가장 신뢰할 수 있는 매칭(해밍 거리가 낮은) 우선 처리

        # 매칭된 특징점의 좌표 계산
        matched_points = []
        for match in matches:
            pt = keypoints[match.trainIdx].pt
            matched_points.append(pt)
            cv2.circle(tracked_frame, (int(pt[0]), int(pt[1])), 3, (0, 255, 0), -1) # 초록색 점

        prev_points = matched_points # 매칭된 좌표 업데이트
```


02

모의 테스트 결과

